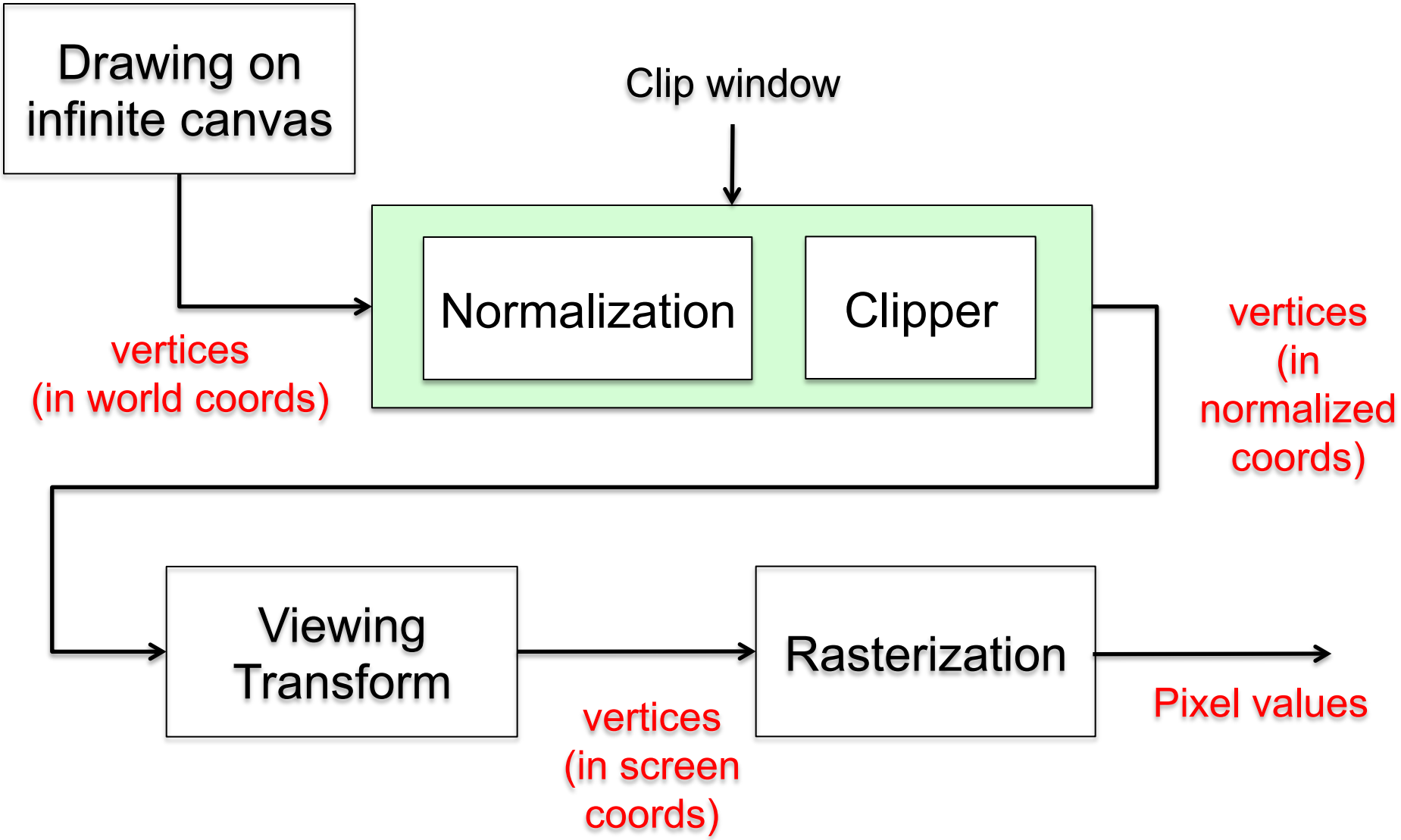
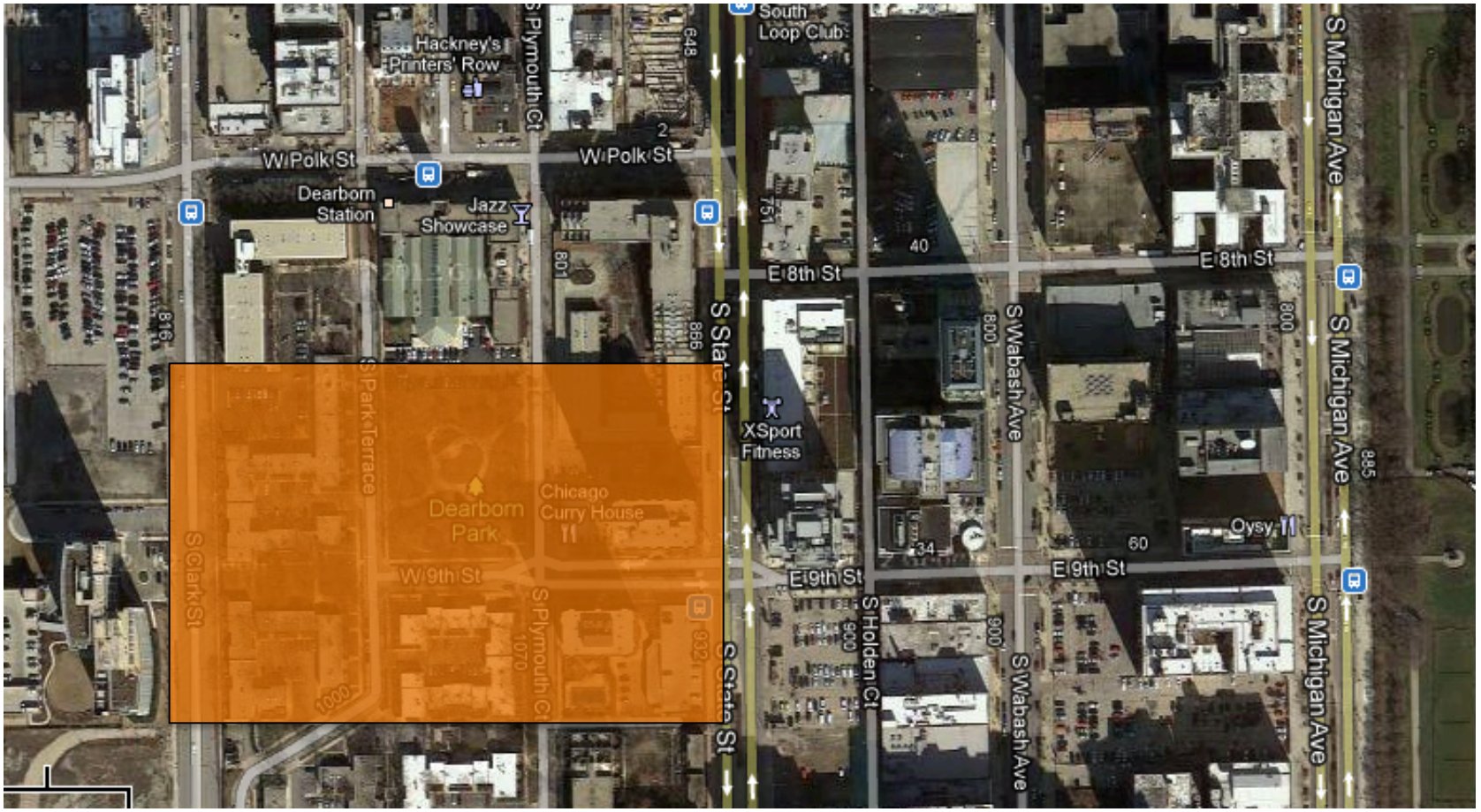
4003-570/4005-761
Computer Graphics 1

2D Viewing and Transformations

2D Pipeline

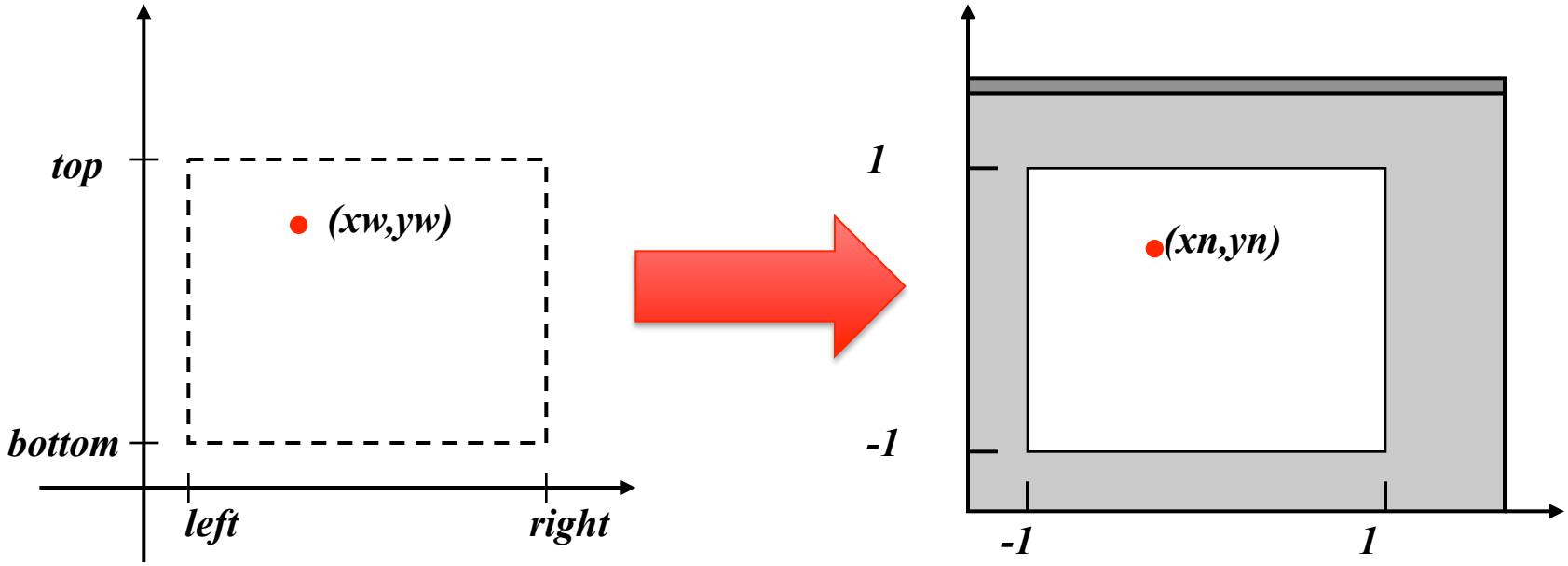

Drawing on infinite canvas
Clip window
Normalization
Clipper
vertices (in world coords)
vertices (in normalized coords)
Viewing Transform
Rasterization
vertices (in screen coords)
Pixel values

Clip Window


Hackney's
Printers' Row
S Plymouth Ct
W Polk St
Dearborn
Station
Jazz
Showcase
South
Loop Club
S Michigan Ave
E 8th St
S Wabash Ave
XSport
Fitness
Dearborn
Park
Chicago
Curry House
W 9th St
E 9th St
S Park Terrace
S Clark St
S Holden Ct
Oysy

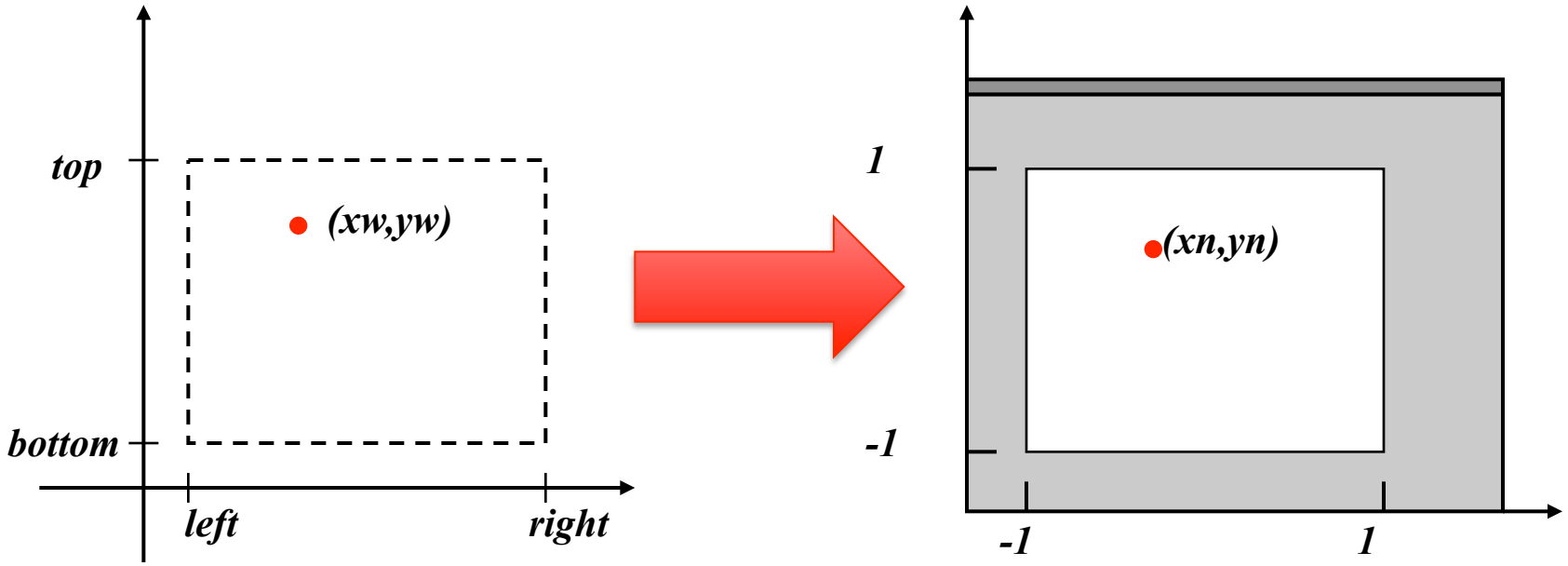
Clipping / Normalization

- World (clipping) window expressed in world coordinates
- Normalized coordinates: center (0,0), 2 units in each direction



Clipping / Normalization

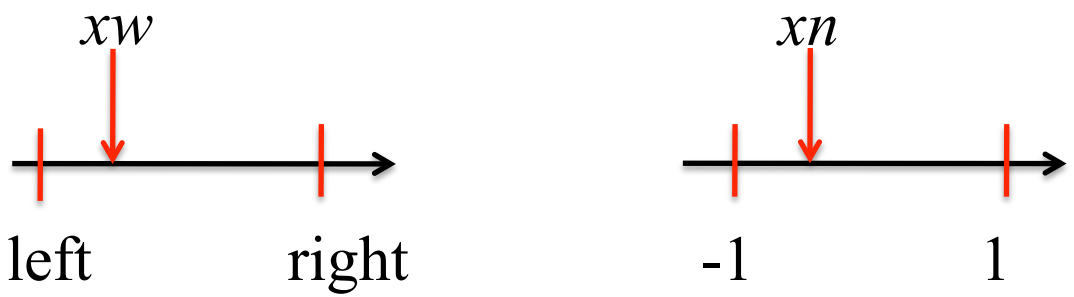
$$xn = Axw + C$$
$$yn = Byw + D$$



Clipping / Normalization

- Goal is to maintain proportions from clip window to normalized window
- For a point (xw, yw) in world space, what is the corresponding point (xn, yn) in normalized space?

$$\frac{(xw - left)}{(right - left)} = \frac{(xn - (-1))}{(1 - (-1))} = \frac{(xn + 1)}{2}$$

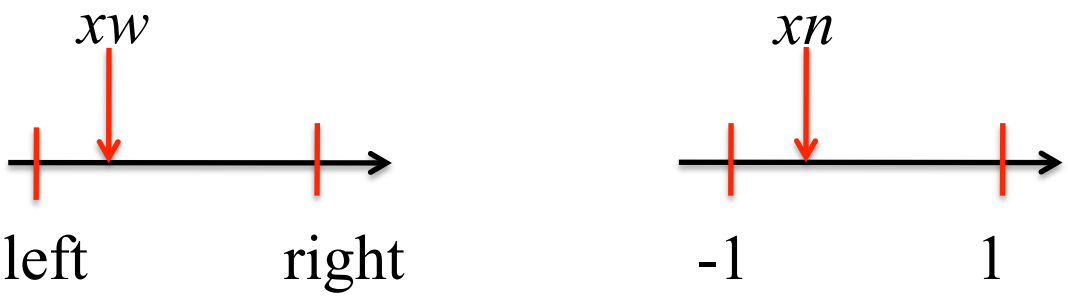


Clipping / Normalization

$$\frac{(xw - left)}{(right - left)} = \frac{(xn - (-1))}{(1 - (-1))} = \frac{(xn + 1)}{2}$$

$$xn = \frac{2(xw - left)}{(right - left)} - 1$$

$$xn = \frac{2}{(right - left)} xw + \left(\frac{-2(left)}{(right - left)} - 1 \right)$$



Clipping / Normalization

$$xn = \frac{2}{(right - left)} xw + \left(\frac{-2(left)}{(right - left)} - 1 \right)$$

A C

$$\boldsymbol{xn = Axw + C}$$

Clipping / Normalization

$$yn = \frac{2}{(top - bottom)} yw + \left(\frac{-2(bottom)}{(top - bottom)} - 1 \right)$$

B D

$$\boldsymbol{yn = Byw + D}$$

Normalization Transform

$$xn = \frac{2}{(right - left)} xw + \left(\frac{-2(left)}{(right - left)} - 1 \right)$$

$$yn = \frac{2}{(top - bottom)} yw + \left(\frac{-2(bottom)}{(top - bottom)} - 1 \right)$$

Normalization Transform

- Can also be found by using basic transformations

Fundamental Concepts

- Transformations are critical to most graphics applications
  - It's important to understand how they work
- Used to alter the size and/or position of things
- Can apply to many image components
  - Objects in scene
  - Camera, lighting, etc.
- We'll start with 2D transformations
  - Easier to understand
  - Same concepts apply to 3D transformations

Representation

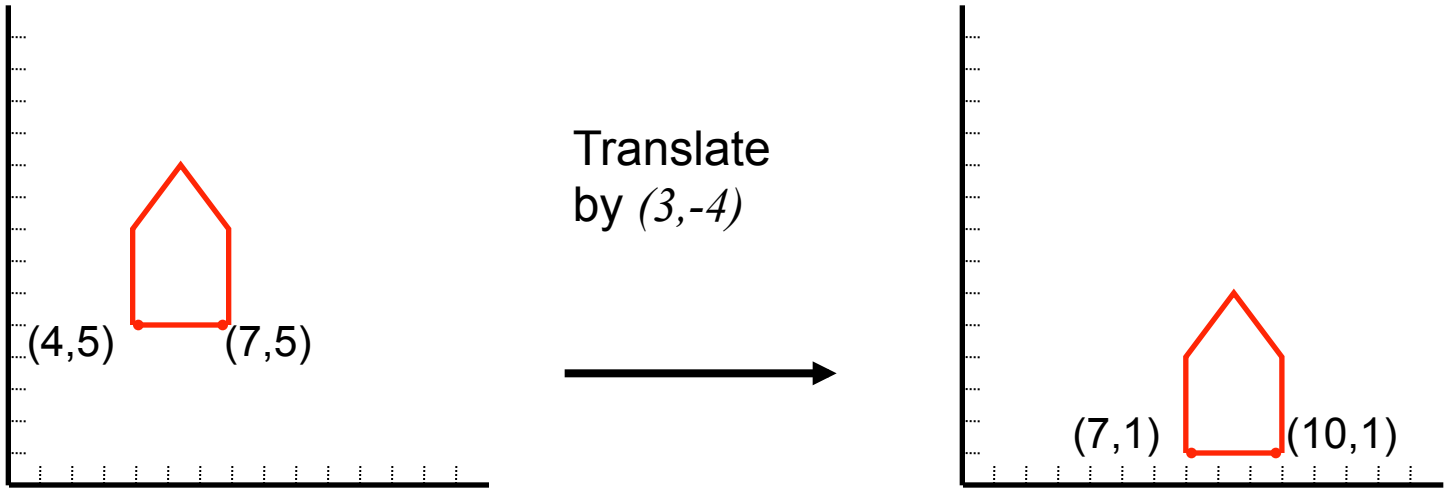
- Most common transformations:
  - *Translation* – moving an object from one position to another
  - *Scaling* – changing the size of an object
  - *Rotation* – revolving an object around a pivot point
- The point $P(x,y)$ is represented as a vector
  - Can be a row vector or a column vector

$$P(x,y) = \begin{bmatrix} x & y \end{bmatrix} \qquad P(x,y) = \begin{bmatrix} x \\ y \end{bmatrix}$$

  - We'll use column vectors
- Using vectors simplifies manipulations

Translation

- Translate vertices by adding an offset to each coordinate
- Translate objects by applying the same translation to each vertex of the object



Translation

- Implemented by addition
- Translation of $P(x,y)$ to $P'(x',y')$ is represented as

$$P' = T + P$$

- We define the translation as

$$x' = x + t_x, \qquad y' = y + t_y$$

- Representing these as vectors,

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \qquad P = \begin{bmatrix} x \\ y \end{bmatrix} \qquad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Scaling

- Implemented with multiplication
- Unlike translation, scaling is relative to the origin
  - Translation is relative to the original position of the object
- Point $P(x,y)$ is scaled to $P'(x',y')$ with the products

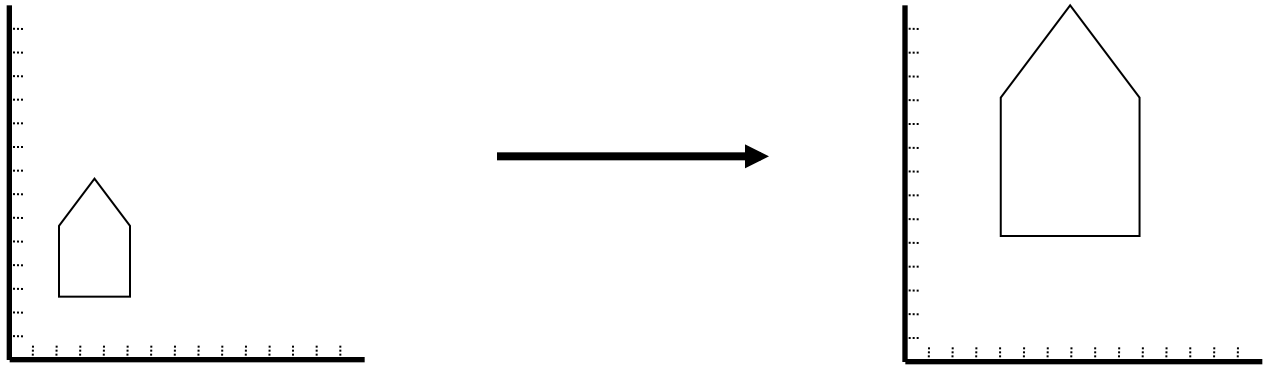
$$x' = s_x \cdot x, \qquad y' = s_y \cdot y$$

- In matrix form: $P' = S \cdot P$

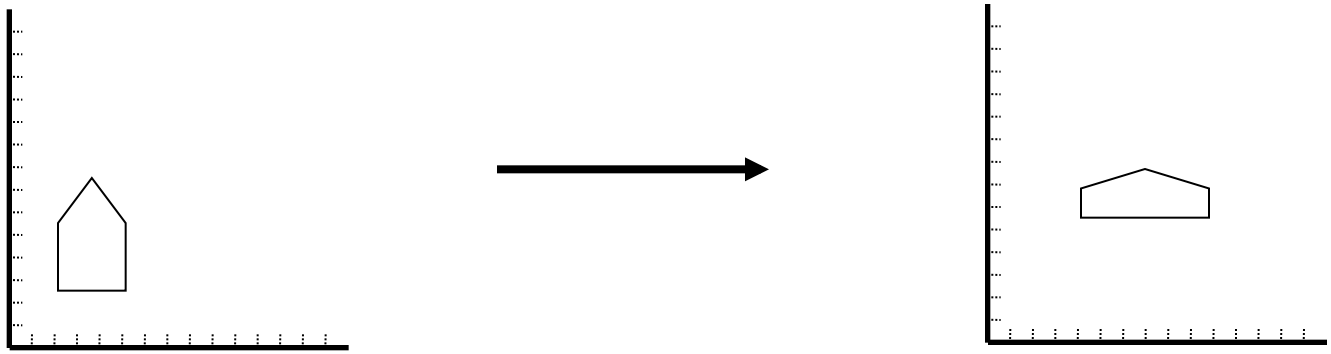
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Scaling

- If $s_x = s_y$, we have a *uniform* scaling:

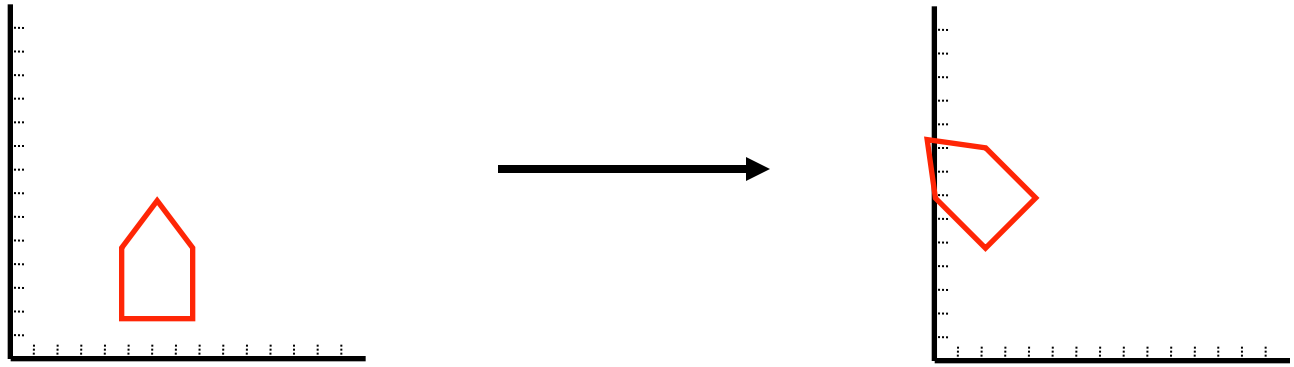


- If s_x != s_y, we have a *differential* scaling:



Rotation

- More complicated
- We rotate points through an angle θ about the origin
  - Positive angles rotate counter-clockwise
- Here is the result of rotating 45° about the origin:

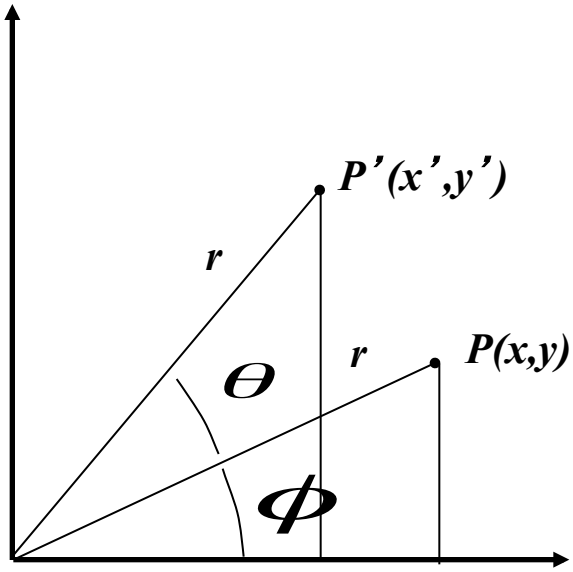


Derivation of Rotation Equations

- Consider points $P(x,y)$ and $P'(x',y')$
- Trigonometry tells us that

$$(P) \quad x = r \cdot \cos\phi$$

$$y = r \cdot \sin\phi$$



- and that

$$(P') \quad x' = r \cdot \cos(\phi + \theta)$$

$$= r \cdot \cos\phi \cdot \cos\theta - r \cdot \sin\phi \cdot \sin\theta$$

$$y' = r \cdot \sin(\phi + \theta)$$

$$= r \cdot \cos\phi \cdot \sin\theta + r \cdot \sin\phi \cdot \cos\theta$$

Rotation Matrices

- Substituting (P) into (P') yields

$$x' = x \cdot \cos\theta - y \cdot \sin\theta = x \cdot \cos\theta + y \cdot (-\sin\theta)$$
$$y' = y \cdot \sin\theta + y \cdot \cos\theta$$

- In matrix form:

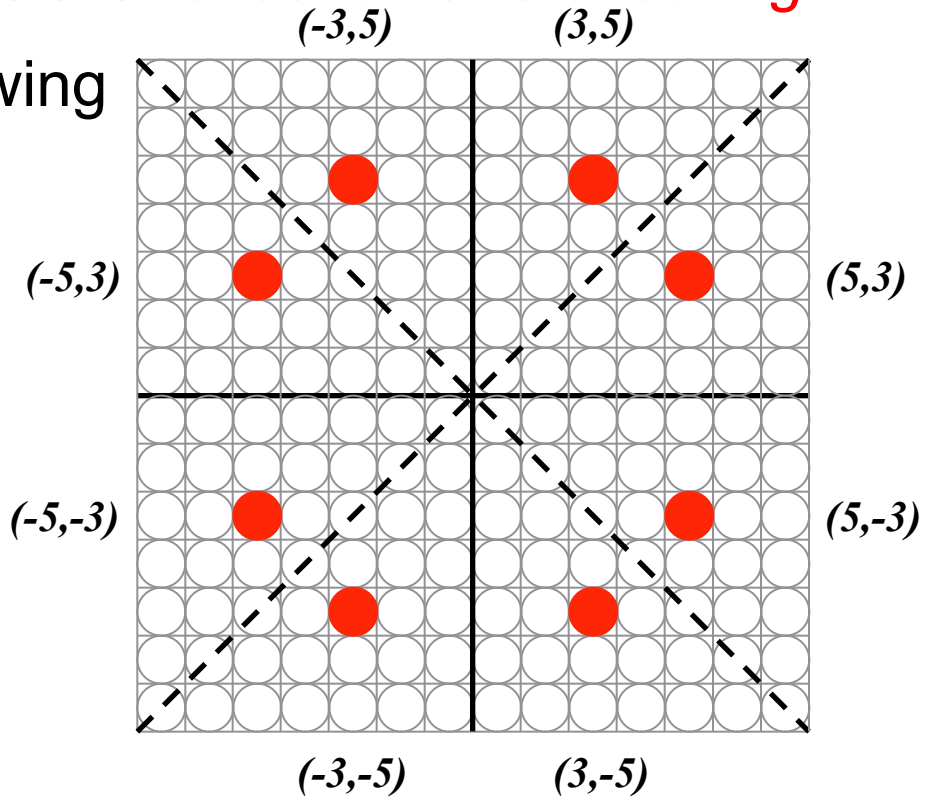
$$P' = R \cdot P$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Affine Transformations

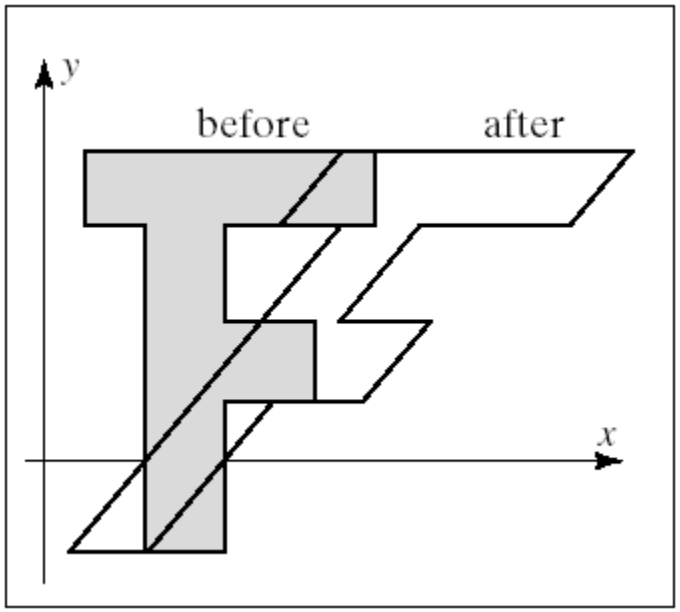
- Translation, scaling, & rotation are *affine* transformations
- Preserve parallelism of lines, but not lengths or angles
- Other affine transformations are *reflection* and *shearing*
- Reflection is useful for drawing symmetric shapes:

  e.g. circles:



Shearing

- We shear by modifying one coordinate
  - Shear in x direction: $x' = x + ay, \quad y' = y$
  - Shear in y direction: $x' = x, \quad y' = bx + y$



Matrix Representations

- Here are the "big three" transformations, in matrix form:

$$P' = T + P$$

$$P' = S \cdot P$$

$$P' = R \cdot P$$

- Unfortunately, translation is different
  - Addition, not multiplication
- Ideally, want to treat them all the same way
  - This will allow us to easily combine different operations

Homogeneous Coordinates

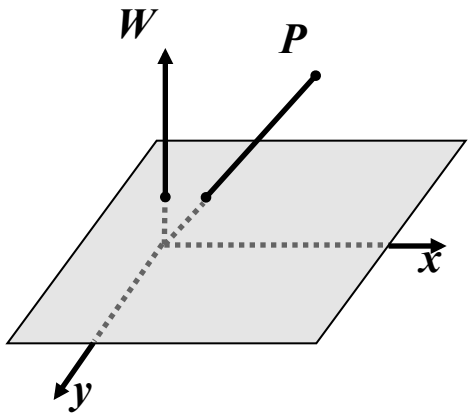
- Solution: use *homogeneous coordinates*
  - First developed in the mid 1940s in geometry
- Allows us to treat all three operations as multiplications
- Basic modification: add a third coordinate to a point
  - Called the *homogeneous parameter*
- The point (x,y) becomes (x_h,y_h,h) where

$$x = \frac{x_h}{h} \qquad y = \frac{y_h}{h}$$

- Another way to write this: $(h \bullet x, h \bullet y, h)$
- Can choose any nonzero h
  - Commonly, we choose 1

Homogeneous Coordinates

- Normally, we use triples to represent points in 3-space
- If we collect all triples (tx, ty, tW) with $t \neq 0$, we get a line



- We homogenize the point by dividing by W
- This gives us the point $(x,y,1)$
- ∴ Homogenized points form the plane defined by $W = 1$

Homogeneous Coordinates

- 2D → 3D Homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- 3D Homogeneous → 2D

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow \begin{bmatrix} x/w \\ y/w \end{bmatrix}$$

Homogeneous Coordinates – Translation

- Points are now three-element vectors
- Transformation matrices must now be 3x3
- This now allows us to use multiplication for translation:

$$P' = T + P \quad \text{becomes} \quad P' = T(t_x, t_y) \cdot P$$

- In matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous Coordinates – Scaling, Rotation

- Scaling and rotation become

$$P' = S(s_x, s_y) \cdot P \qquad P' = R(\theta) \cdot P$$

- In matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous Coordinates – Reflection

- Around the x axis:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Around the y axis:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Around the diagonals:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous Coordinates – Shearing

- Shearing is a bit different
- We shear by modifying one coordinate
  - Shear in x: $x' = x + ay, \quad y' = y$
  - Shear in y: $x' = x, \quad y' = bx + y$

SHx

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & a & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

SHy

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ b & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Inverse Transformations – Translation

- Sometimes we need to invert or reverse a transformation
- We construct an inverse transformation matrix by "inverting" the original transformation
- For translation, we negate the translation distances:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \qquad T^{-1} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Inverse Transformations – Scaling

- For scaling, we use the reciprocals of the scale factors:

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 & 0 \\ 0 & \frac{1}{s_y} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Inverse Transformations – Rotation

- For rotation, we negate the angle:

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad R^{-1} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Note that this is the transpose of R

Caution

- Some texts use *row* vectors for points
  - We're using *column* vectors
- They also alter the multiplication sequence to *premultiply* by row vectors rather than *postmultiplying* by column vectors:

$$P \cdot M = P' \qquad P' = M \cdot P$$

- We must perform a transposition to switch forms:

$$(P \cdot M)^T = M^T \cdot P^T$$

Multiple Translations

- Intuitively, we know that translating a point twice is equivalent to a single, combined translation

$$P' = T(d_{x1}, d_{y1}) \cdot P \quad \text{then} \quad P'' = T(d_{x2}, d_{y2}) \cdot P'$$

- results in

$$\begin{aligned} P'' &= T(d_{x2}, d_{y2}) \cdot T(d_{x1}, d_{y1}) \cdot P \\ &= T(d_{x1} + d_{x2}, d_{y1} + d_{y2}) \cdot P \end{aligned}$$

$$\begin{bmatrix} 1 & 0 & d_{x2} \\ 0 & 1 & d_{y2} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & d_{x1} \\ 0 & 1 & d_{y1} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d_{x1} + d_{x2} \\ 0 & 1 & d_{y1} + d_{y2} \\ 0 & 0 & 1 \end{bmatrix}$$

Multiple Scaling

- The same is true for scaling (multiplicative, not additive)

$$P' = S(s_{x1}, s_{y1}) \cdot P \quad \text{then} \quad P'' = S(s_{x2}, s_{y2}) \cdot P'$$

- results in

$$P'' = S(s_{x2}, s_{y2}) \cdot S(s_{x1}, s_{y1}) \cdot P$$
$$= S(s_{x1} \cdot s_{x2}, s_{y1} \cdot s_{y2}) \cdot P$$

$$\begin{bmatrix} s_{x2} & 0 & 0 \\ 0 & s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_{x1} & 0 & 0 \\ 0 & s_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} s_{x2} \times s_{x1} & 0 & 0 \\ 0 & s_{y2} \times s_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Multiple Rotation

- Rotation, like translation, is additive

$$P' = R(\theta) \cdot P \quad \text{then} \quad P'' = R(\phi) \cdot P'$$

- results in $P'' = R(\phi) \cdot R(\theta) \cdot P$

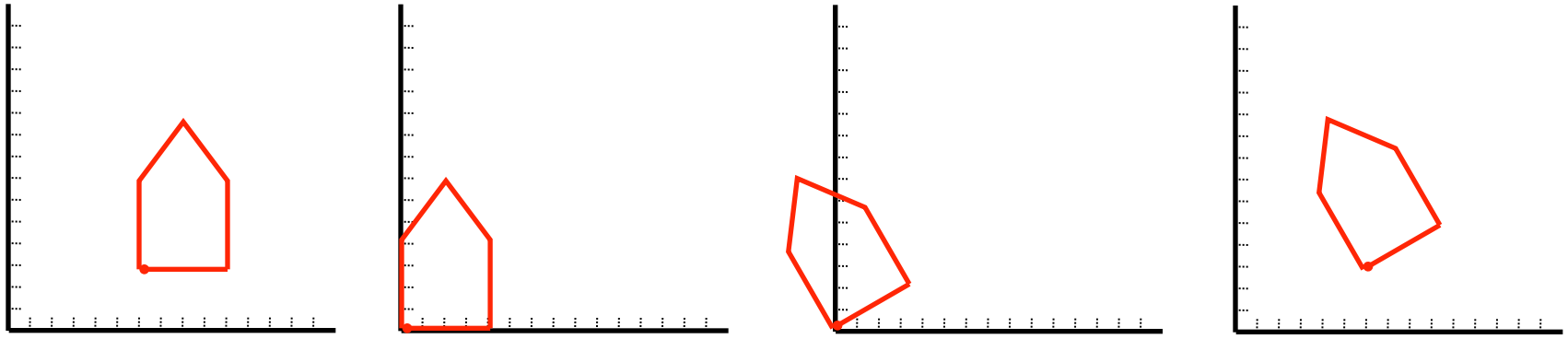
$$= R(\theta + \phi) \cdot P$$

$$\begin{bmatrix} \cos\phi & -\sin\phi & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos(\theta+\phi) & -\sin(\theta+\phi) & 0 \\ \sin(\theta+\phi) & \cos(\theta+\phi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Composition of Transformations

- We can combine a series of transformations into a composite transformation
- Recall that rotation is about the origin
- To rotate about an arbitrary point P, must:
  - Translate P to the origin
  - Rotate
  - Translate the origin back to P



Composition of Transforms – Rotation

- To translate $P1(x_1,y_1)$ to the origin, use $T(-x_1,-y_1)$
- We then rotate this using $R(\theta)$
- Finally, we translate back by $T(x_1,y_1)$
- The full sequence: $T(x_1,y_1) \cdot R(\theta) \cdot T(-x_1,-y_1)$

$$= \begin{bmatrix} 1 & 0 & x_1 \\ 0 & 1 & y_1 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta & x_1(1-\cos\theta) + y_1 \times \sin\theta \\ \sin\theta & \cos\theta & y_1(1-\cos\theta) - x_1 \times \sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

Composition of Transforms – Scaling

- Scaling is also relative to the origin
- To scale relative to an arbitrary point, we translate that point to the origin, scale by $S(s_x,s_y)$, and translate back
- The full sequence: $T(x_1,y_1) \cdot S(s_x,s_y) \cdot T(-x_1,-y_1)$

$$= \begin{bmatrix} 1 & 0 & x_1 \\ 0 & 1 & y_1 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} s_x & 0 & x_1\left(1-s_x\right) \\ 0 & s_y & y_1\left(s_y\right) \\ 0 & 0 & 1 \end{bmatrix}$$

Rigid-Body Transformation

- Also called a *rigid-motion* transformation
- Uses only rotation and translation
- Maintains size and shape of object
- General form:

$$\begin{bmatrix} r_{xx} & r_{xy} & t_x \\ r_{yx} & r_{yy} & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Efficiency

- Consider the general rigid-body transformation matrix:

$$\begin{bmatrix} r_{xx} & r_{xy} & t_x \\ r_{yx} & r_{yy} & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- Applying this does nine multiplications and six additions

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} r_{xx} & r_{xy} & t_x \\ r_{yx} & r_{yy} & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

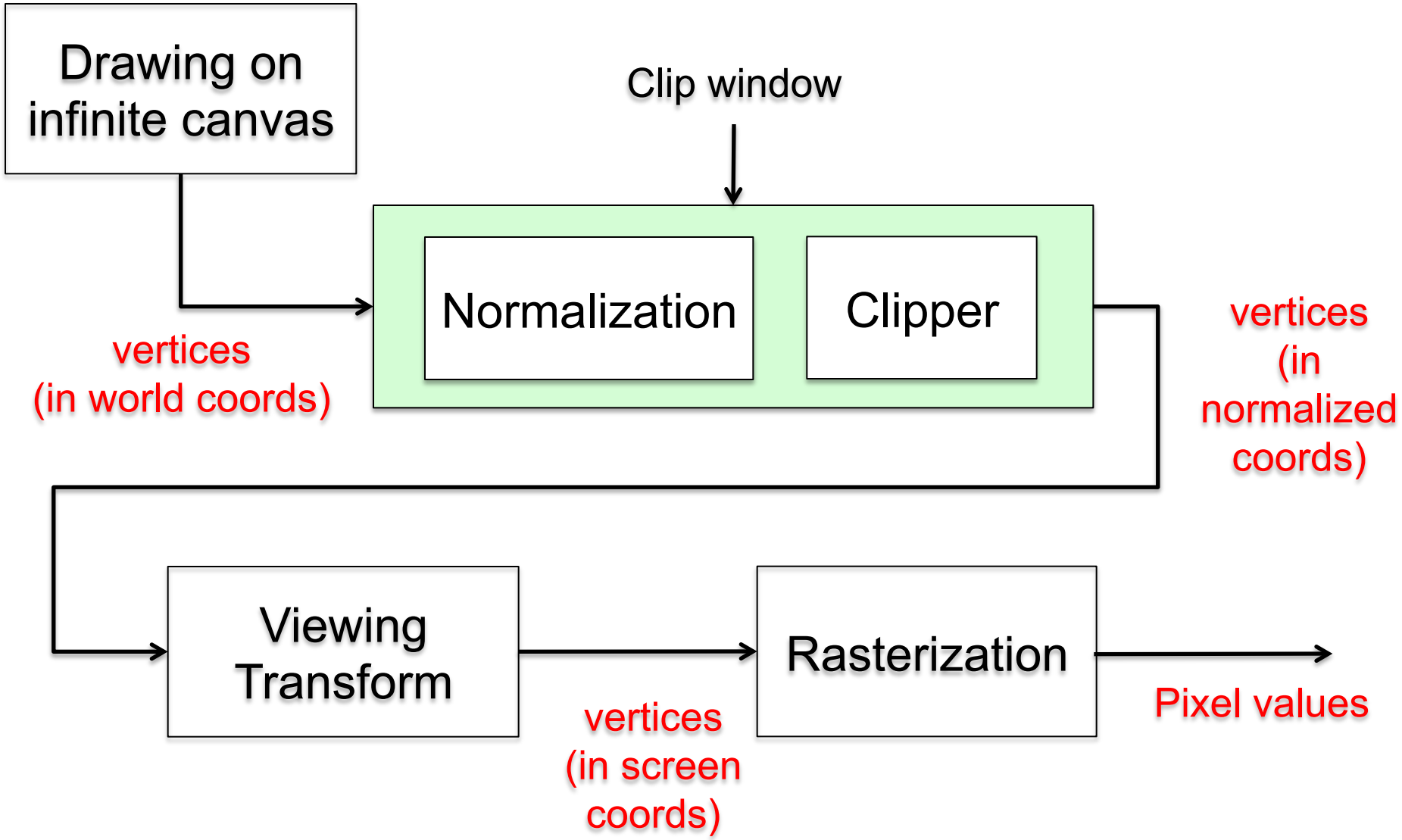
Efficiency

- Can save time by noting the structure of the matrix
  - The bottom row, [0 0 1], always yields 1 as the bottom coordinate of the resulting point
  - The rightmost column entries are always multiplied against 1, the bottom coordinate of the original point
- Can eliminate five multiplications and two additions:
- $$x' = r_{xx} \cdot x + r_{xy} \cdot y + t_x$$
- $$y' = r_{yx} \cdot x + r_{yy} \cdot y + t_y$$
- Very significant
- May be applying this to thousands of vertices

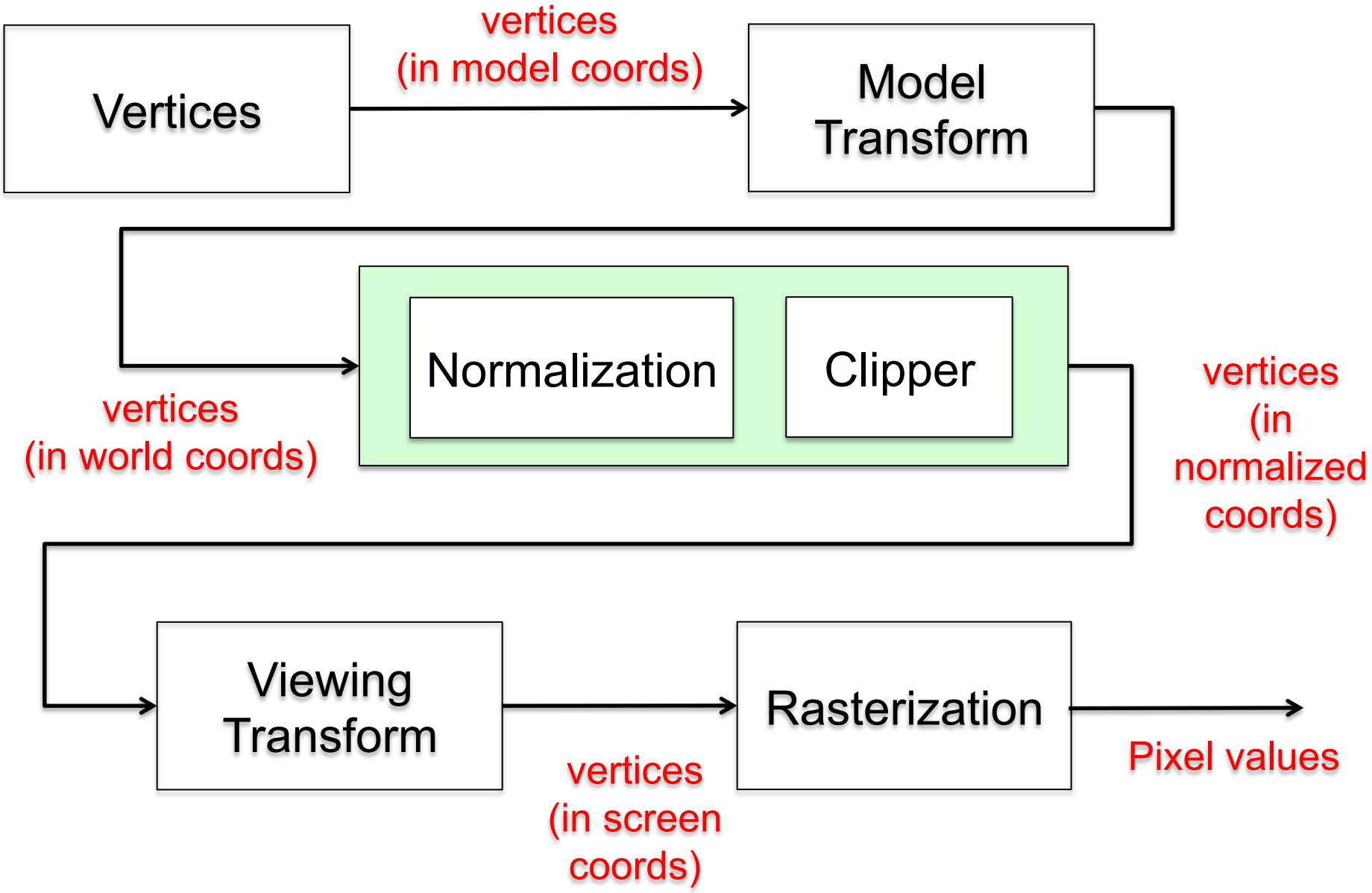
Model vs. World Coordinates

- Benefit of transformations:
  - Objects can be defined in their own coordinate system
  - Use transformations to “place” onto the world canvas
  - Sequence of transformations represented by a single matrix
  - Why?
    - “Object” centric modeling
    - Multiple instances of same object
    - Model once, place many times

2D Pipeline

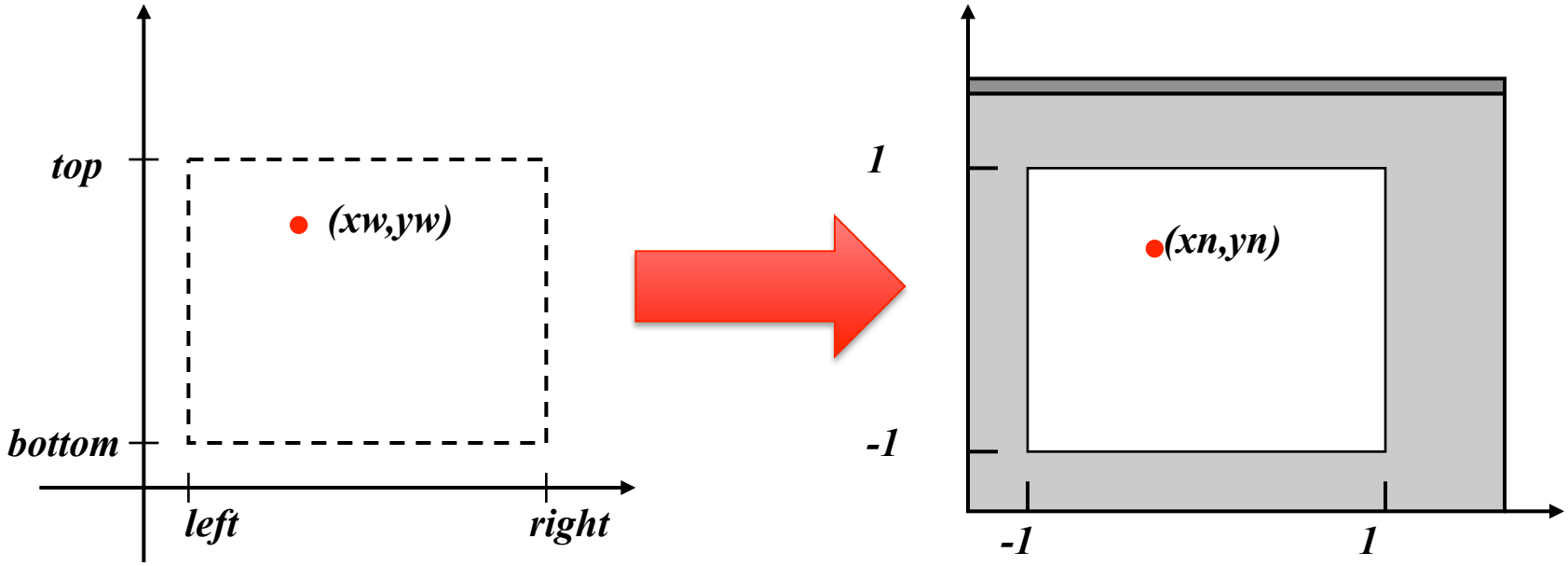

Drawing on infinite canvas
Clip window
Normalization
Clipper
vertices (in world coords)
vertices (in normalized coords)
Viewing Transform
Rasterization
vertices (in screen coords)
Pixel values

2D Pipeline


Vertices
vertices
(in model coords)
Model
Transform
vertices
(in world coords)
Normalization
Clipper
vertices
(in
normalized
coords)
Viewing
Transform
vertices
(in screen
coords)
Rasterization
Pixel values

Clipping / Normalization

- World (clipping) window expressed in world coordinates
- Normalized coordinates: center (0,0), 2 units in each direction



Normalization Transform – Direct approach

$$xn = \frac{2}{(right - left)} xw + \left(\frac{-2(left)}{(right - left)} - 1 \right)$$

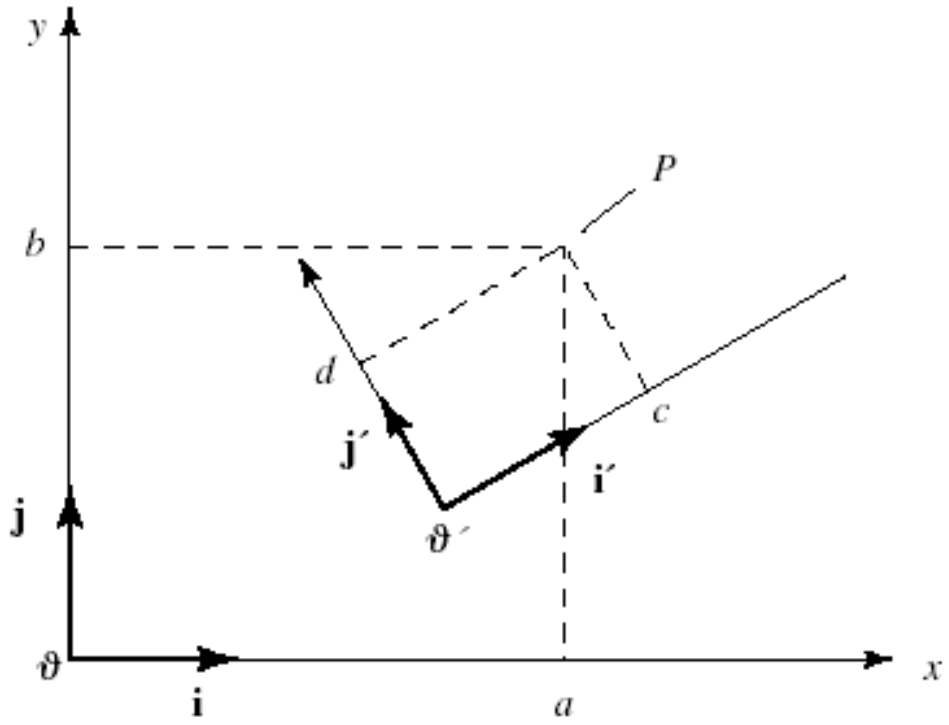
$$yn = \frac{2}{(top - bottom)} yw + \left(\frac{-2(bottom)}{(top - bottom)} - 1 \right)$$

Normalization Transform – In Matrix form

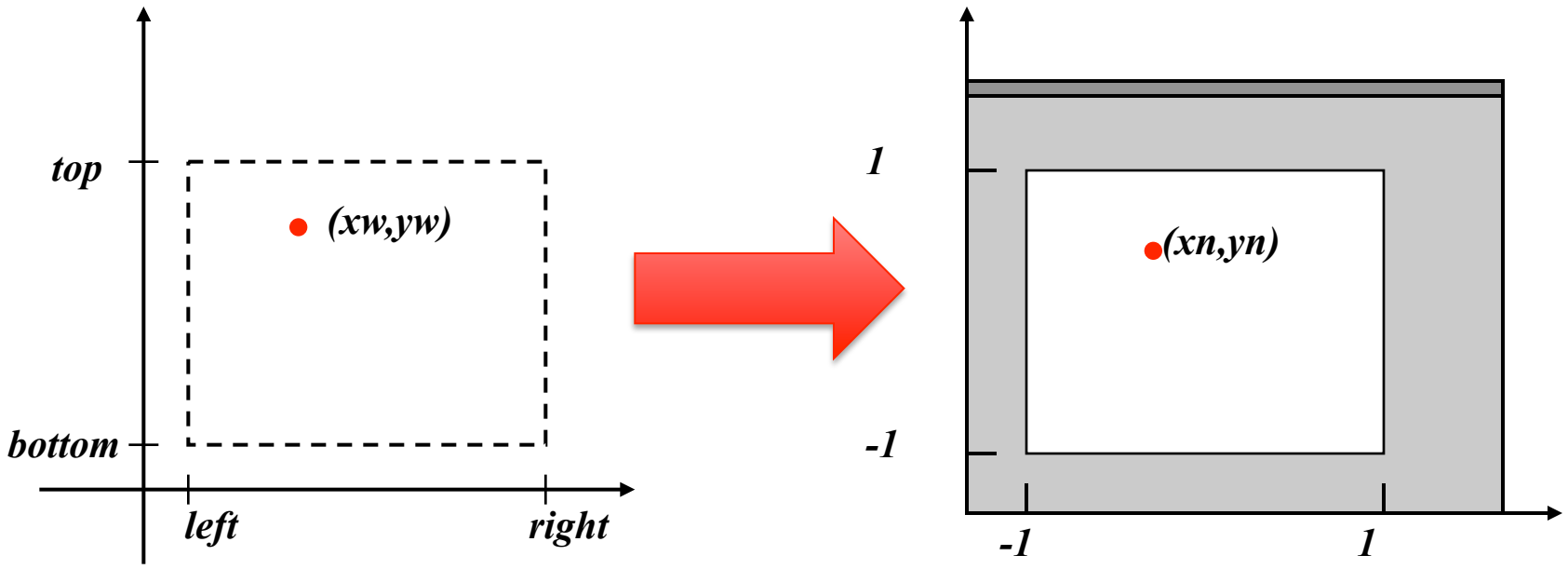
$$\begin{bmatrix} \frac{2}{(right-left)} & 0 & \frac{-2(left)}{(right-left)} - 1 \\ 0 & \frac{2}{(top-bottom)} & \frac{-2(bottom)}{(top-bottom)} - 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Coordinate Transformations

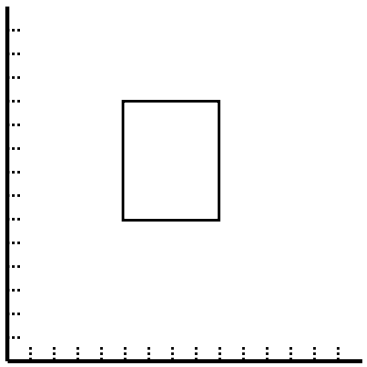
- Alternate view:
  - Point exists within a coordinate system
  - Applying a transformation changes the *coordinate system* being used, not the *coordinates within the current coordinate system*



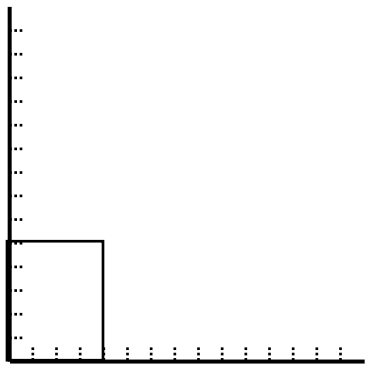
Clipping / Normalization



Clipping / Normalization

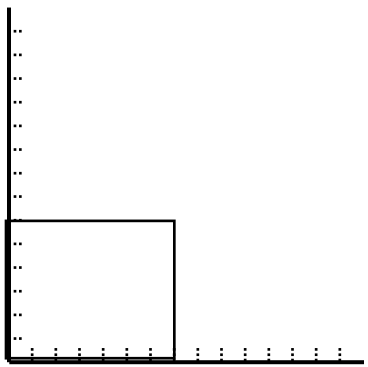


Clip window



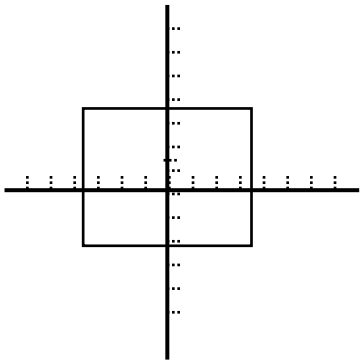
Translate to origin

T (-left, -bottom)



Perform scaling

?

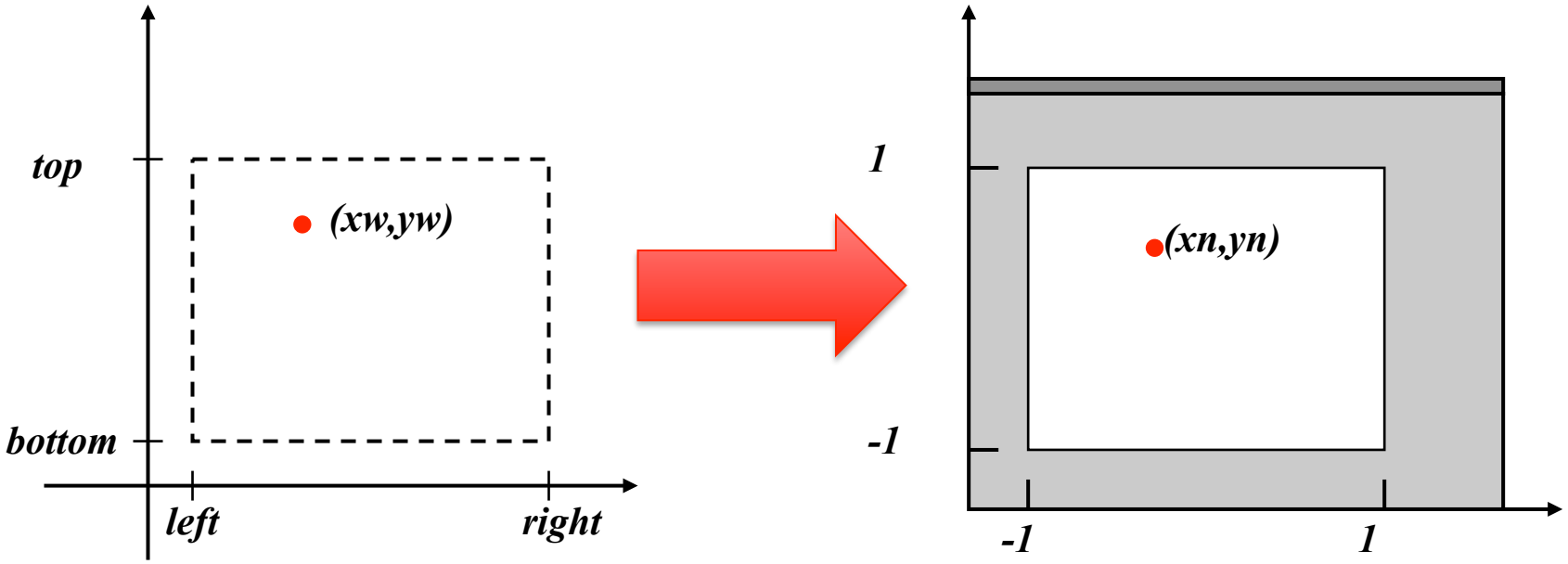


Translate to position of normalized window

T (-1.0, -1.0)

Clipping / Normalization

- Scaling values are the ratios of the dimensions



$$s_x = \frac{2}{(right - left)} \qquad s_y = \frac{2}{(top - bottom)}$$

Clipping / Normalization

- The sequence:

$$T(-1\ ,\ -1) \cdot S(s_x, s_y) \cdot T(-left, -bottom)$$

- The matrices:

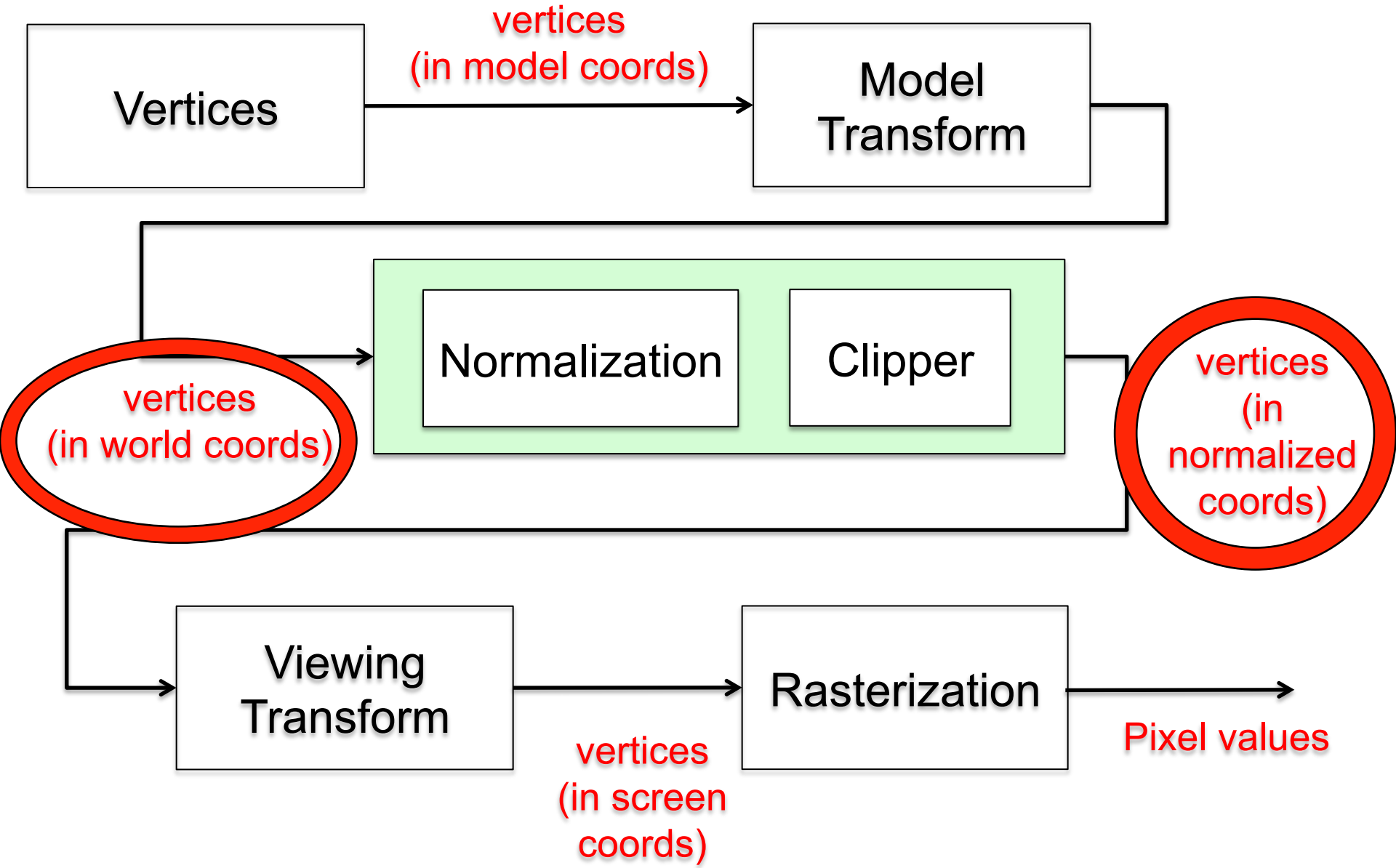
$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} \frac{2}{(left - right)} & 0 & 0 \\ 0 & \frac{2}{(top - bottom)} & 0 \\ 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & -left \\ 0 & 1 & -bottom \\ 0 & 0 & 1 \end{bmatrix}$$

Normalization Transform – In Matrix form

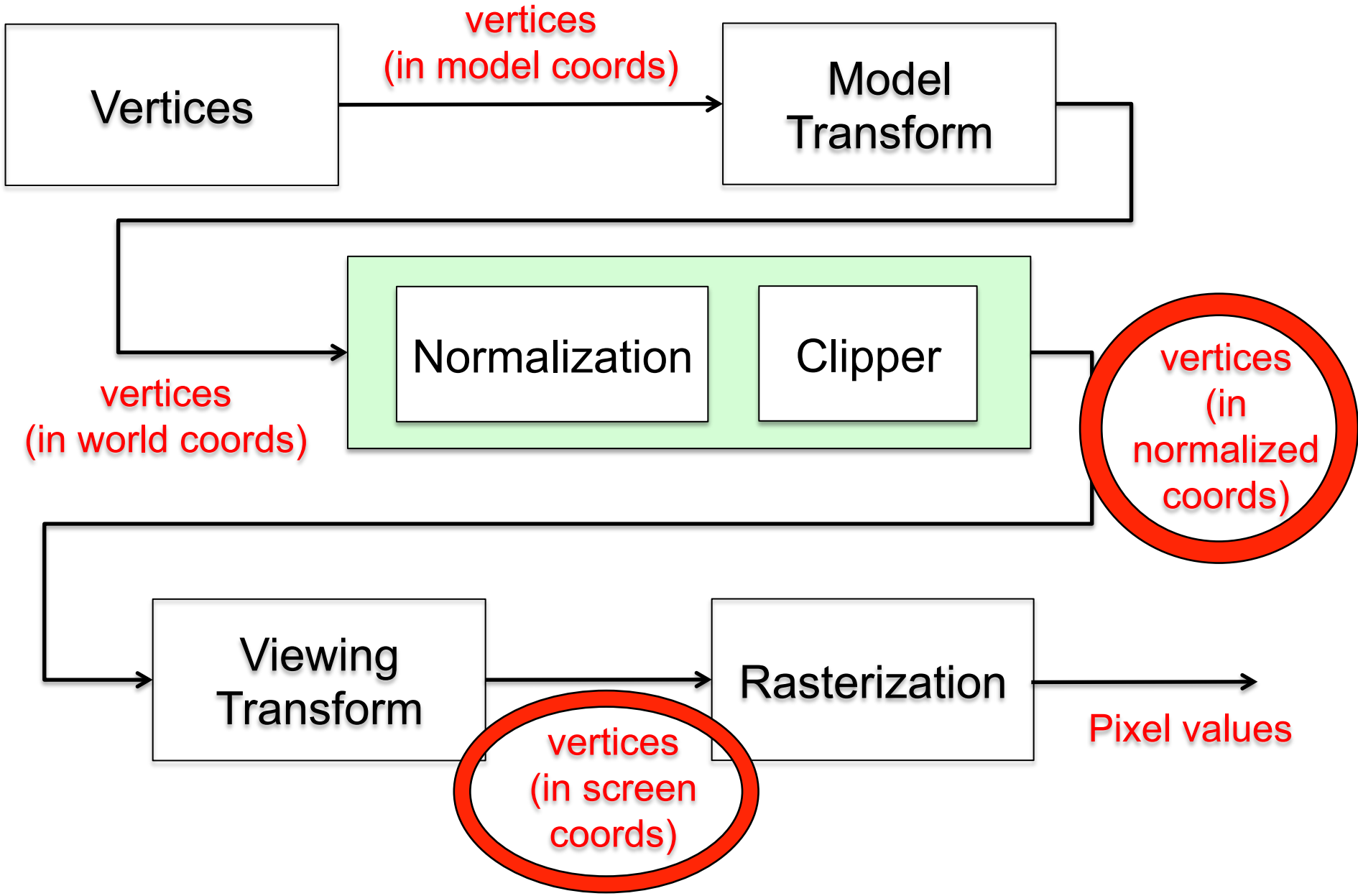
- Matches what we derived earlier

$$\begin{bmatrix} \frac{2}{(right-left)} & 0 & \frac{-2(left)}{(right-left)} - 1 \\ 0 & \frac{2}{(top-bottom)} & \frac{-2(bottom)}{(top-bottom)} - 1 \\ 0 & 0 & 1 \end{bmatrix}$$

2D Pipeline

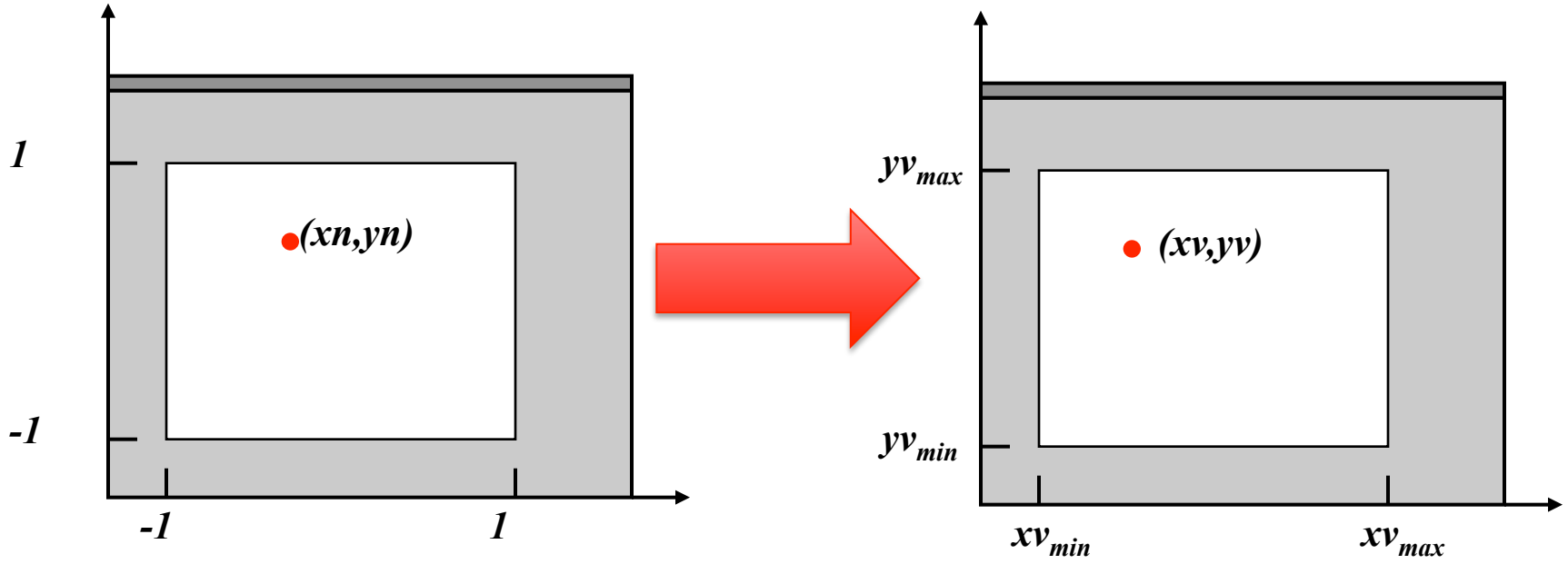

Vertices
vertices
(in model coords)
Model
Transform
Normalization
Clipper
vertices
(in world coords)
vertices
(in
normalized
coords)
Viewing
Transform
Rasterization
vertices
(in screen
coords)
Pixel values

2D Pipeline

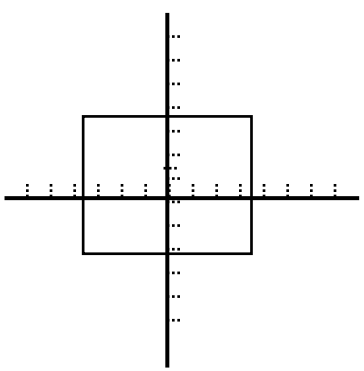

Vertices
vertices
(in model coords)
Model
Transform
vertices
(in world coords)
Normalization
Clipper
vertices
(in
normalized
coords)
Viewing
Transform
vertices
(in screen
coords)
Rasterization
Pixel values

View Transform (from normalized coordinates)

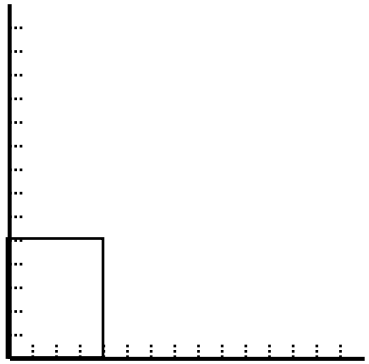
- Normalized coordinates: (0,0) in center, 2 units in each direction
- Display window expressed in screen coordinates



View Transform (from normalized coordinates)

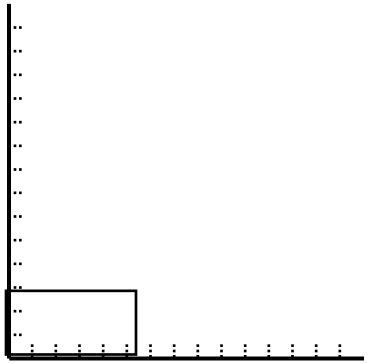


Normalized window



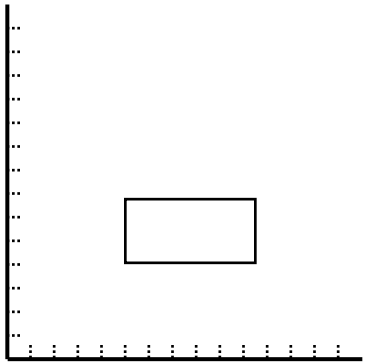
Translate to origin

T (1, 1)



Perform scaling

?

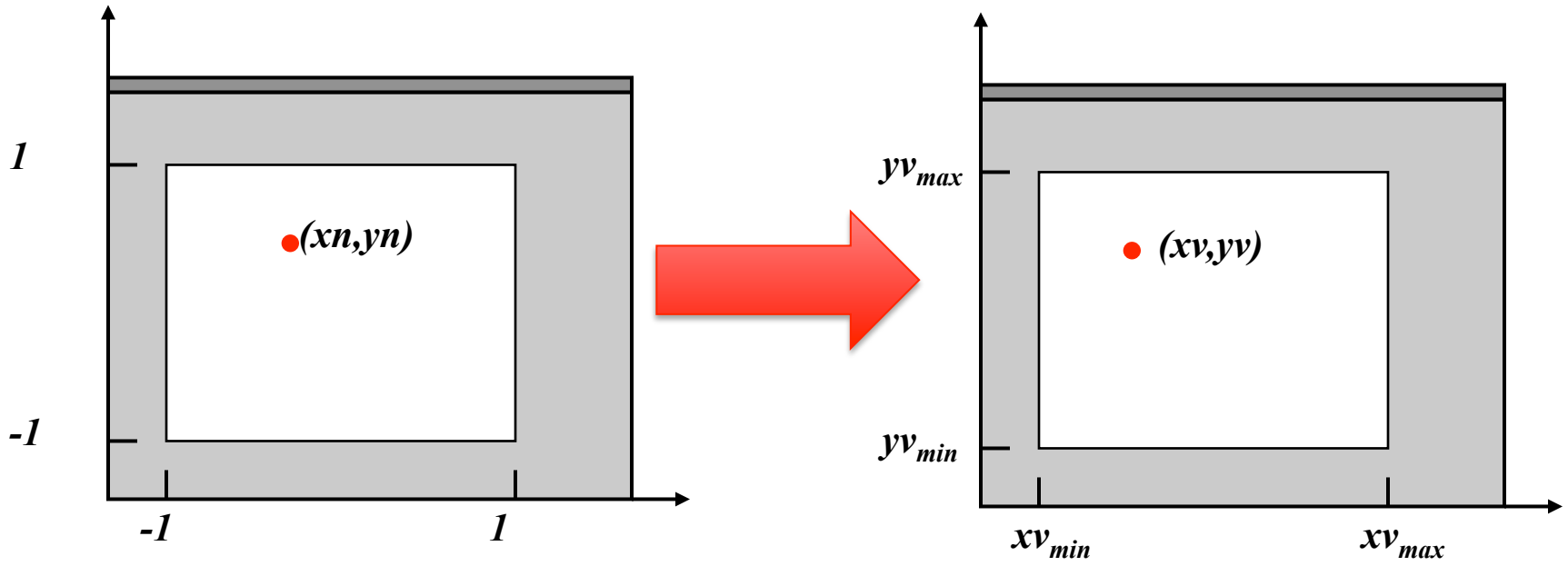


Translate to position of view window

T (xmin, ymin)

View Transform (from normalized coordinates)

- Scaling values are the ratios of the dimensions



$$s_x = \frac{(xv_{\max} - xv_{\min})}{2}$$

$$s_y = \frac{(yv_{\max} - yv_{\min})}{2}$$

View Transform (from normalized coordinates)

- The sequence:

$$T(xmin, ymin) \cdot S(s_x, s_y) \cdot T(1,1)$$

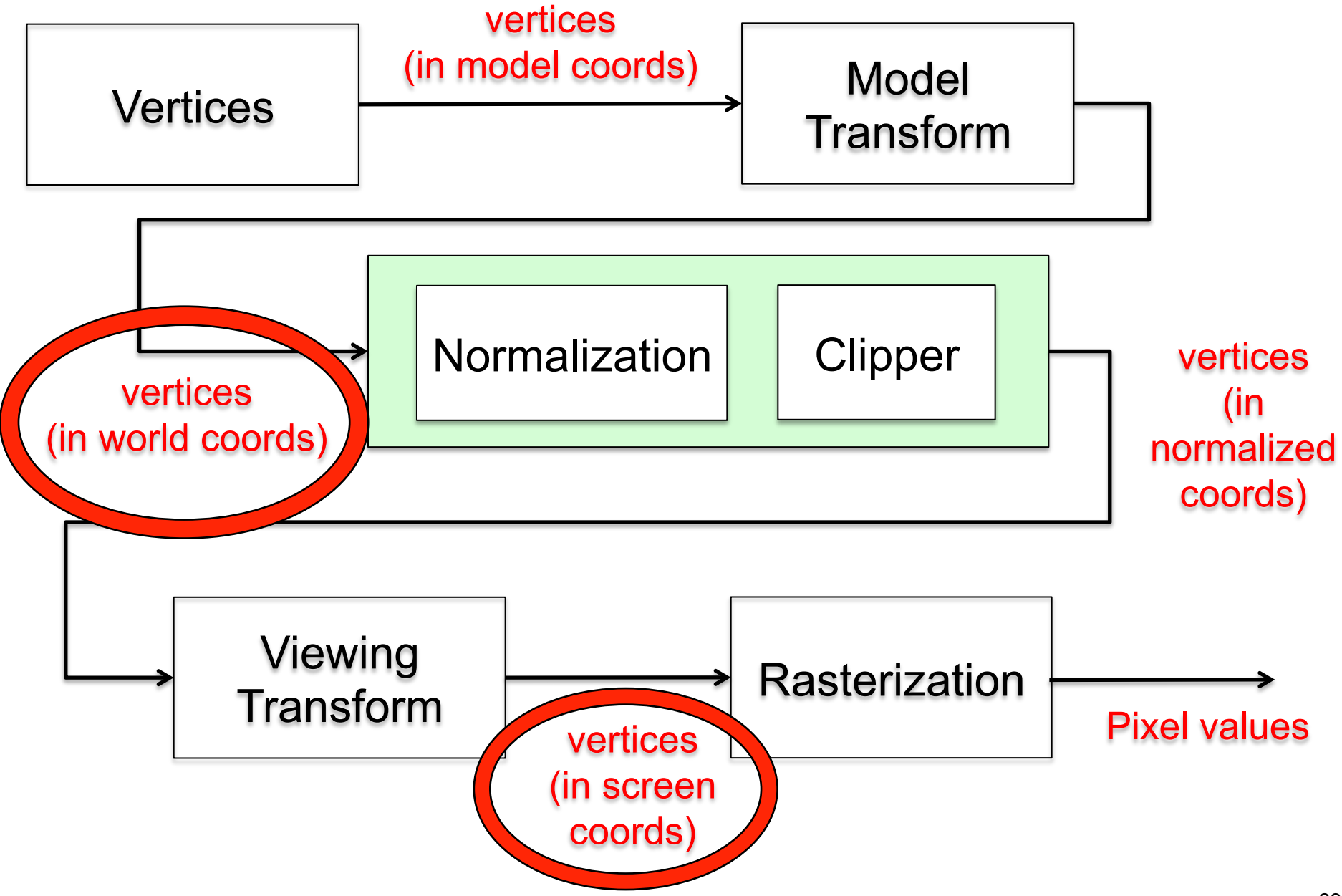
- The matrices:

$$\begin{bmatrix} 1 & 0 & xv_{\min} \\ 0 & 1 & yv_{\min} \\ 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} \frac{(xv_{\max} - xv_{\min})}{2} & 0 & 0 \\ 0 & \frac{(yv_{\max} - yv_{\min})}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

View Transform (from normalized coordinates)

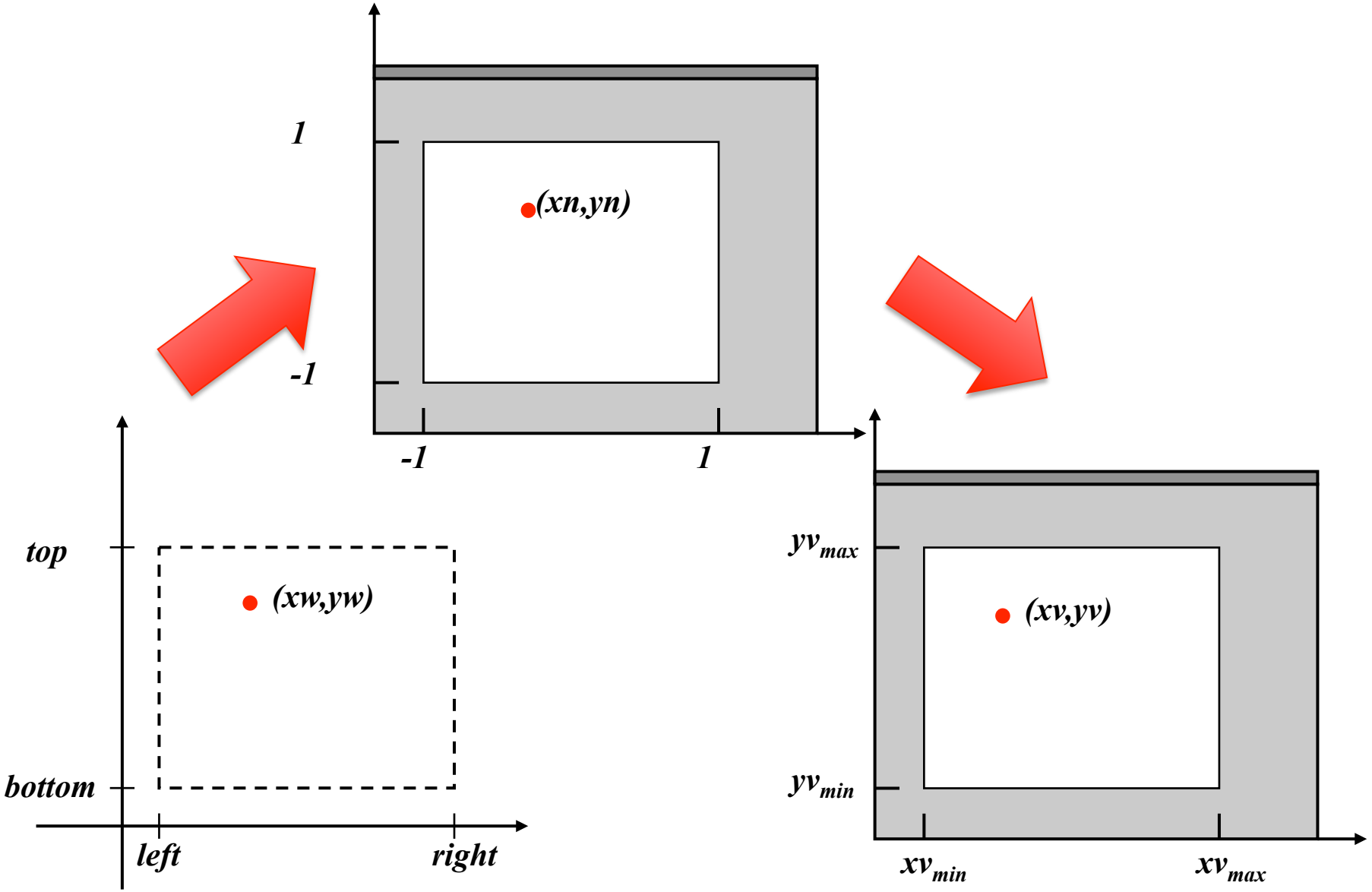
$$\begin{bmatrix} \frac{(xv_{\max} - xv_{\min})}{2} & 0 & \frac{(xv_{\max} + xv_{\min})}{2} \\ 0 & \frac{(yv_{\max} - yv_{\min})}{2} & \frac{(yv_{\max} + yv_{\min})}{2} \\ 0 & 0 & 1 \end{bmatrix}$$

2D Pipeline


Vertices
vertices
(in model coords)
Model
Transform
vertices
(in world coords)
Normalization
Clipper
vertices
(in
normalized
coords)
Viewing
Transform
Rasterization
Pixel values
vertices
(in screen
coords)

View Transform (from clip window)

- World -> Normalized -> Screen



View Transform (from clip window)

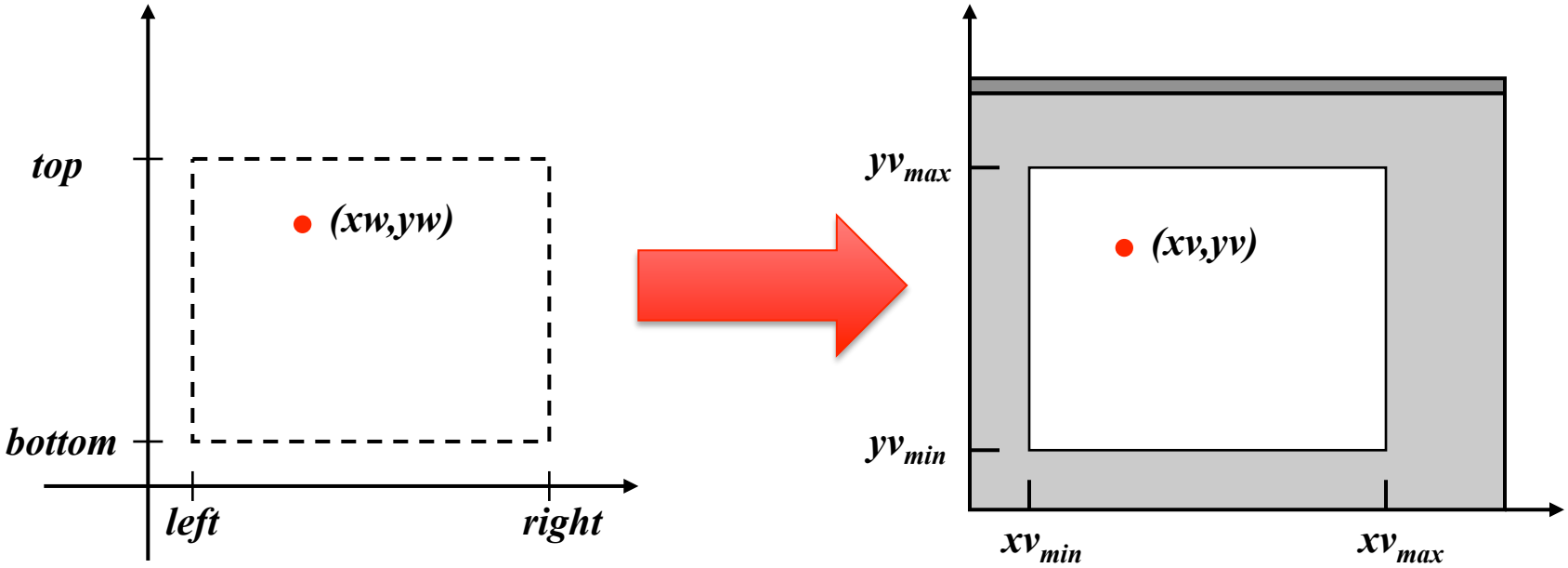
$$\begin{bmatrix} \frac{(xv_{max} - xv_{min})}{2} & 0 & \frac{(xv_{max} + xv_{min})}{2} \\ 0 & \frac{(yv_{max} - yv_{min})}{2} & \frac{(yv_{max} + yv_{min})}{2} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \frac{2}{(right - left)} & 0 & \frac{-2(left)}{(right - left)} - 1 \\ 0 & \frac{2}{(top - bottom)} & \frac{-2(bottom)}{(top - bottom)} - 1 \\ 0 & 0 & 1 \end{bmatrix}$$

View Transform (from Normalized) Matrix

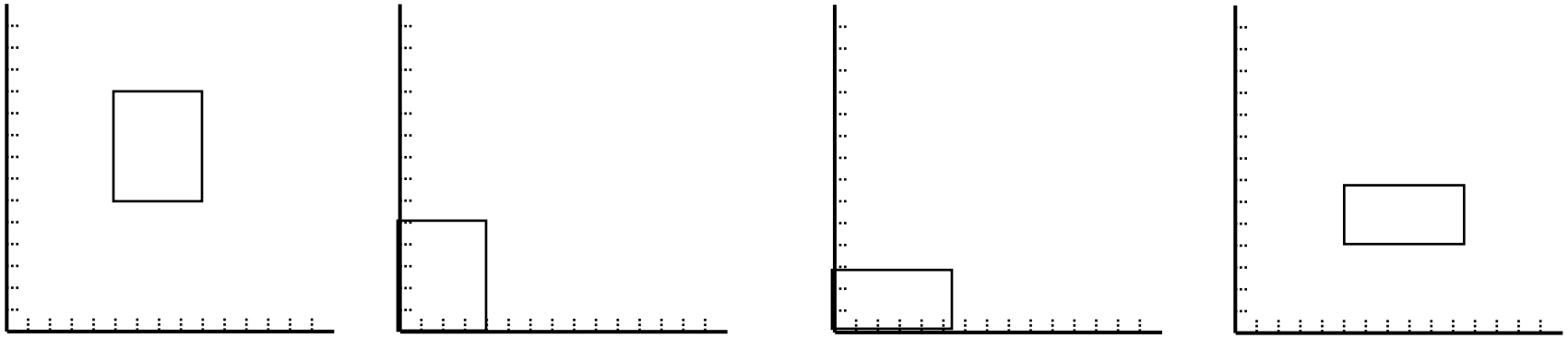
Normalization Matrix

View Transform (from clip window)

- World (clipping) window expressed in world coordinates
- Display window expressed in screen coordinates



View Transform (from clip window)



Clip window

Translate to origin

T (-left, -bottom)

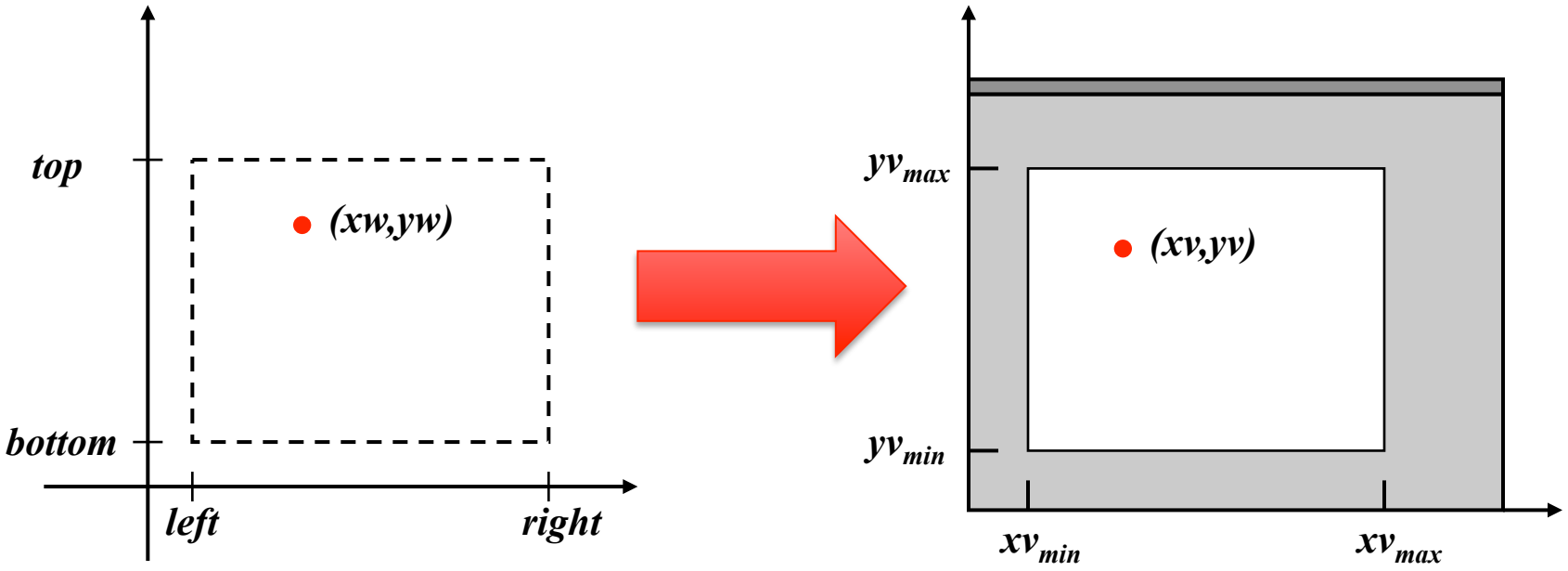
Perform scaling

Translate to position of view window

T (xmin, ymin)

View Transform (from clip window)

- Scaling values are the ratios of the dimensions



$$s_x = \frac{(xv_{\max} - xv_{\min})}{(right - left)} \qquad s_y = \frac{(yv_{\max} - yv_{\min})}{(top - bottom)}$$

View Transform (from clip window)

- The sequence:

$$T(xmin, ymin) \cdot S(s_x, s_y) \cdot T(-left, -bottom)$$

- The matrices:

$$\begin{bmatrix} 1 & 0 & xv_{\min} \\ 0 & 1 & yv_{\min} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \frac{(xv_{\max} - xv_{\min})}{(right - left)} & 0 & 0 \\ 0 & \frac{(yv_{\max} - yv_{\min})}{(top - bottom)} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -left \\ 0 & 1 & -bottom \\ 0 & 0 & 1 \end{bmatrix}$$

View Transform (from clip window)

$$\begin{bmatrix} s_x & 0 & t_x \\ 0 & s_y & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$s_x = \frac{xv_{\max} - xv_{\min}}{right - left} \qquad s_y = \frac{yv_{\max} - yv_{\min}}{top - bottom}$$

$$t_x = \frac{((right)(xv_{\min})) - ((left)(xv_{\max}))}{right - left} \qquad t_y = \frac{((top)(yv_{\min})) - ((bottom)(yv_{\max}))}{top - bottom}$$